Efficient 3D Reconstruction in One Minute via Gaussian Splatting

Hengxing Liu
Tianjin University
Tianjin, China
chrisliu.jz@gmail.com

Xiaojie Guo*
Tianjin University
Tianjin, China
xj.max.guo@gmail.com

Abstract

We present a rapid 3D Gaussian Splatting reconstruction pipeline, tailored for the SIGGRAPH Asia 3DGS Fast Reconstruction Challenge, with the goal of reconstructing scenes within a strict one-minute time limit. Given the challenge's COLMAP-format input data, we developed a fast reconstruction method featuring two key contributions: (1) a dynamic resolution planning strategy based on DashGaussian, and (2) a multi-view consistency-guided splitting and pruning mechanism for Gaussian primitives, inspired by FastGS. These techniques collectively enable photorealistic scene reconstruction in under a minute. Our approach achieved 3rd place in the challenge, delivering a PSNR of 27.25 with an average reconstruction time of 34 seconds.

CCS Concepts

• **Computing methodologies → Computer graphics**.

Keywords

3D Gaussian Splatting, Efficient Reconstruction

ACM Reference Format:
Hengxing Liu and Xiaojie Guo. 2025. Efficient 3D Reconstruction in One Minute via Gaussian Splatting. In *Craig-Boris '25: Craig's test event for Boris LaTeX Code 2 (Craig-Boris '25), January 1–5, 2025, TBa, TAS, USA.* ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3563045.3563055

1 Introduction

3D Gaussian Splatting (3DGS)[Kerbl et al. 2023] has rapidly established itself as a premier representation for neural scene reconstruction, offering a compelling alternative to implicit volumetric methods like NeRF. By explicitly modeling scenes with anisotropic 3D Gaussian primitives rather than relying on expensive ray-marching and dense sampling, 3DGS enables high-speed differentiable rasterization. This explicit nature facilitates real-time rendering at high frame rates, a critical capability for latency-sensitive applications in VR/AR, robotics, and digital twins, where interactive photorealism directly impacts user immersion and system utility. Despite these rendering advantages, the reconstruction latency remains a significant bottleneck. Optimizing a 3DGS scene involves iteratively refining millions of parameters—spanning position, covariance, and spherical harmonics coefficients. Standard pipelines typically require minutes to hours to converge, heavily burdened by the optimization overhead on high-resolution inputs. This computational cost prohibits on-the-fly reconstruction scenarios, such as online mapping or instant asset generation, where rapid adaptation is as crucial as final visual quality. Bridging the gap between real-time rendering and real-time training is thus a pivotal challenge in the field. In this work, we argue that accelerating the convergence of the training phase is key to unlocking practical, time-constrained reconstruction. We specifically target the SIGGRAPH Asia 3DGS Fast Reconstruction Challenge, proposing a pipeline that balances aggressive optimization speed with high-fidelity output.

*Corresponding Author.

Craig-Boris '25, TBa, TAS, USA

ACM ISBN /25/01
https://doi.org/10.1145/3563045.3563055

2 Method

2.1 Background: 3D Gaussian Splatting

The 3D Gaussian Splatting framework reconstructs a scene by initializing a collection of 3D primitives from a sparse point cloud, typically obtained via Structure-from-Motion (SfM)[Schonberger and Frahm 2016]. The scene is modeled as a set of Gaussians:

$$\{G_i(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\mathbf{p}_i)^T \Sigma_i^{-1}(\mathbf{x}-\mathbf{p}_i)}\}_{i=1}^{P}, \tag{1}$$

where P denotes the total count of primitives. Each primitive G_i is explicitly parameterized by a mean position $\mathbf{p}_i \in \mathbb{R}^3$, a world-space covariance matrix $\Sigma_i \in \mathbb{R}^{3\times3}$, a view-dependent color vector $\mathbf{c}_i$ (often represented via spherical harmonics), and a scalar opacity $o_i \in (0, 1)$.

Novel views are synthesized by projecting these 3D Gaussians onto the image plane. The final pixel color $C(\mathbf{u})$ at coordinate $\mathbf{u}$ is derived through front-to-back alpha blending, accumulating the contributions of overlapping 2D projected Gaussians G_i^{2D}:

$$C(\mathbf{u}) = \sum_{i=1}^{P} T_i(\mathbf{u}) \cdot w_i(\mathbf{u}) \cdot \mathbf{c}_i, \quad T_i(\mathbf{u}) = \prod_{j=1}^{i-1}(1 - w_j(\mathbf{u})), \tag{2}$$

where the blending weight is given by $w_i(\mathbf{u}) = o_i G_i^{2D}(\mathbf{u})$. Since this rasterization pipeline is fully differentiable, the parameters of G_i can be refined by minimizing the photometric error between the rendered output and the ground-truth training images.

2.2 Frequency Guided Resolution Scheduler

Following the strategy proposed in DashGaussian[Chen et al. 2025], we employ a Frequency-Guided Resolution Scheduler for progressive resolution control. Considering an image $\mathbf{I} \in \mathbb{R}^{H\times W}$ and a downsampled version $\mathbf{I}_r \in \mathbb{R}^{H/r \times W/r}$, we extract their frequency maps with Discrete Fourier Transform (DFT): $\mathbf{F} = \mathrm{DFT}(\mathbf{I})$ and $\mathbf{F}_r = \mathrm{DFT}(\mathbf{I}_r)$. The critical task is to partition the total training steps S appropriately between varying resolutions. This decision is driven by a significance score that quantifies information content at each level:

$$\chi(\mathbf{F}) = \frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{h}\sum_{j=1}^{w}\|\mathbf{F}^n_{i,j}\|_2, \tag{3}$$

where $\mathbf{F}^n = \mathrm{DFT}(\mathbf{I}^n)$, and $\{\mathbf{I}^n\}_{n=1}^{N}$ represents the set of training views at resolution (h, w). This definition places particular emphasis on high-frequency numerical features. The logic is straightforward: if the gap $\chi(\mathbf{F}) - \chi(\mathbf{F}_r)$ is substantially larger than the $\chi(\mathbf{F}_r)$, the majority of optimization should occur at the higher resolution. We quantify this distribution using a fraction function:

$$f(\mathbf{F}, \mathbf{F}_r) = \frac{\chi(\mathbf{F}) - \chi(\mathbf{F}_r)}{\chi(\mathbf{F})}, \tag{4}$$

which assigns $S \cdot f(\mathbf{F}, \mathbf{F}_r)$ iterations to high-resolution rendering. Accordingly, we will switch resolution from $(H_r, W_r)/r$ to (H, W) at iteration s_r, where $s_r = S \cdot (1 - f(\mathbf{F}, \mathbf{F}_r)) = S \cdot \chi(\mathbf{F}_r)/\chi(\mathbf{F})$.

In practical applications, this approach generalizes to multiple intermediate resolutions. Assuming a set of monotonically increasing downsampling factors $r_1, r_2, \ldots, r_m$ for images $\mathbf{I}_{r_1}, \ldots, \mathbf{I}_{r_m}, \mathbf{I}$ are progressively involved for supervision throughout the optimization process. The rendering resolution is increased from $(H/r_i, W/r_i)$ to $(H/r_{i-1}, W/r_{i-1})$ at the s_{r_i} iteration.

2.3 Multi-view Consistent Densification And Pruning

Inspired by FastGS[Ren et al. 2025], we utilize a multi-view consistent strategy for the densification and pruning of Gaussian primitives. By calculating the loss between rendered multi-view images and the ground truth, we assess the influence of each Gaussian primitive. This metric serves as the criterion for deciding when to split, clone, or prune primitives. Specifically, given K camera views $V = \{v_j\}_{j=1}^{K}$, we compute the error between the rendered color $r^j_{u,v}$ and the ground-truth color $g^j_{u,v}$ at pixel (u, v) for each view v_j:

$$e^j_{u,v} = \frac{1}{C}\sum_{c=1}^{C}|r^{j,c}_{u,v} - g^{j,c}_{u,v}|, \tag{5}$$

where $c \in (1, 2, \ldots, C)$ denotes the color channel. We then construct the loss map $\mathcal{M}^j \in \mathbb{R}^{w \times h}$ from the per-pixel errors:

$$\mathcal{M}^j = \mathcal{N}\left(\{e^j_{u,v}\}_{u=0,v=0}^{w-1,h-1}\right), \tag{6}$$

where $\mathcal{N}$ denotes a min–max normalization function. A threshold τ is then applied to $\mathcal{M}^j$ to identify pixel with high reconstruction error, forming a mask:

$$\mathcal{M}^j_{\text{mask}} = \mathbb{I}(\mathcal{M}^j > \tau), \tag{7}$$

where pixels P with $\mathcal{M}^j_{mask}(u, v) = 1$ indicate regions of poor reconstruction quality.

Next, we need to find the Gaussian primitives associated with these higher error pixels. Each primitive is mapped to the 2D image plane to determine its footprint Ω. We then apply an indicator

(a) DashGaussian (b) ours (c) GT

Figure 1: Qualitative comparison on challenge dataset.

function $\mathbb{I}(\mathcal{M}^l_{mask}(p) = 1)$ to flag pixels with high error values. An importance score, s_d, is calculated for every Gaussian primitive by accumulating the high-error pixels within its footprint across all views and averaging the result:

$$s_d = \frac{1}{K}\sum_{j=1}^{K}\sum_{p}^{\Omega}\mathbb{I}(\mathcal{M}^j_{mask}(p) = 1). \tag{8}$$

A larger value of s_d implies that the Gaussian primitive falls into problematic regions, suggesting it should be prioritized for densification.

Meanwhile, we calculate the photometric loss for each view $v^j \in V$ by comparing the rendered image r^j against the ground truth g^j:

$$E^j_{\text{photo}} = (1 - \lambda)L^j_1 + \lambda(1 - L^j_{\text{SSIM}}). \tag{9}$$

Given that photometric loss reliably reflects reconstruction fidelity, we utilize it to define a pruning score for each primitive:

$$s_p = \mathcal{N}\left(\sum_{j=1}^{K}\left(\sum_{p}^{\Omega}\mathbb{I}(M^j_{\text{mask}}(p) = 1)\right)\cdot E^j_{\text{photo}}\right). \tag{10}$$

Here s_p can be interpreted as a quantitative measure of the contribution of the Gaussian primitive to the degradation of the overall rendering quality. A Gaussian primitive is selected for pruning if its pruning score s_p exceeds a predefined threshold τ_p, indicating that it has relatively low contribution to rendering quality across multiple views.

3 Experiment

We conduct all experiments on an RTX 4090 GPU and Intel(R) Xeon(R) Platinum 8358P CPU. The maximum number of training iterations is 6,000. In this experiment, DashGaussian[Chen et al. 2025] serves as the baseline. We incorporated a multi-view consistency-based densification and pruning strategy into the implementation. Table.1 presents a performance comparison between our proposed method and the baseline on the challenge dataset. As shown in the table.1, the introduction of the multi-view consistency strategy leads to a significant reduction in the number of Gaussian primitives and a decrease in reconstruction time, with only a marginal drop in PSNR. To balance reconstruction quality and efficiency, we opted to retain this strategy. Please note that the reconstruction times reported in this paper differ from those on the official leaderboard, likely due to differences in CPU specifications. Figure.1 illustrates the visual results of our method and provides a qualitative comparison with the baseline.

Table 1: The quantitative results on challenge dataset.

Exp	PSNR(dB)	Time(s)	N^{GS}
DashGaussian	27.31	45.41	571614
ours	27.25	42.75	363440

References

Youyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. 2025. DashGaussian: Optimizing 3D Gaussian Splatting in 200 Seconds. In *CVPR*.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans. Graph.* 42, 4 (2023), 139:1–139:14.

Shiwei Ren, Tianci Wen, Yongchun Fang, and Biao Lu. 2025. FastGS: Training 3D Gaussian Splatting in 100 Seconds. *arXiv preprint arXiv:2511.04283* (2025).

Johannes L. Schonberger and Jan-Michael Frahm. 2016. Structure-from-Motion Revisited. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. doi:10.1109/cvpr.2016.445